5 HEURES / 3 MR / 8 TESTS

PortPilot

Votre premier scanner TCP de laboratoire · Python, tests et trois merge requests GitLab

Le défi en 5 heures

Écrire vous-même un outil qui tente une connexion TCP à quelques ports de votre machine, conserve leur état et produit un bilan. Livrer progressivement le programme avec des tests qui bloquent réellement une fusion en cas d'échec.

Un petit outil, une vraie chaîne de livraison

- Vous connaissez déjà les variables, les listes et les fonctions. Vous allez pratiquer les boucles, les conditions, les exceptions, le module socket, les dictionnaires et les assertions.
- Vous écrivez le programme, ses huit fonctions de test et la configuration CI. Cet énoncé donne des contrats et des pistes de documentation, sans code Python ni YAML de solution.
- Trois MR successives : validation → connexion → scan et bilan. Chacune ajoute une fonctionnalité, conserve les tests précédents et fusionne après un pipeline vert au dernier commit.

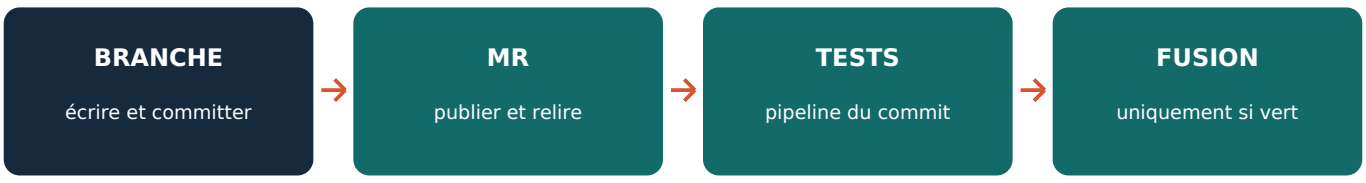
Le cadre du laboratoire

La cible reste exactement **127.0.0.1**. Aucun paramètre de cible distante. Entre 1 et 10 ports distincts, une connexion à la fois, un essai par port et un délai fixe de 0,5 seconde. Aucune donnée applicative envoyée, aucune bannière récupérée.

Un port ouvert signifie seulement qu'une connexion TCP a réussi. Cela ne prouve ni une vulnérabilité ni la nature du service. L'exercice concerne uniquement votre laboratoire local autorisé.

À préparer avant le chronomètre

Python 3, Git, un éditeur, le compte sur le GitLab de l'école, un runner utilisable et les droits nécessaires aux règles de fusion. Sans runner ou règles accessibles, vous pouvez coder et tester localement, mais l'objectif de CI bloquante reste non validé.



Pipeline rouge : fusion bloquée. Corriger le code, puis vérifier le nouveau vert.

01 · Préparer GitLab

0–25 min · Le verrou de fusion doit être en place avant la première MR.

Créer ou rejoindre le dépôt

Utilisez l'instance et l'URL fournies par l'école. Créez un projet personnel ou de cours autorisé, privé sauf consigne contraire. Initialisez seulement README.md et .gitignore sur la branche principale. Le fichier .gitignore doit exclure les caches Python et portpilot.zip avant le premier code.

Configurer la branche principale

Dans **Settings → Repository → Branch rules / Protected branches** (selon la version), un Maintainer, Owner ou responsable protège la branche principale. Respectez les règles déjà fixées par l'école.

- **Allowed to push and merge = No one** : aucun push direct. Désactivez le force push.
- **Allowed to merge = Developers + Maintainers** si vous devez fusionner ; sinon Maintainers, avec un responsable disponible pour les trois fusions.

Activer le blocage par les tests

Dans **Settings → Merge requests → Merge checks**, activez **Pipelines must succeed**. Laissez **Skipped pipelines are considered successful** désactivé quand l'option existe. Une MR sans pipeline ne valide pas l'exercice.

Une preuve observable

Pour vérifier le blocage CI, utilisez un compte autorisé à fusionner, une MR non-Draft et sans conflit. Une interdiction liée au rôle du compte ne prouve pas que les tests bloquent la fusion.

Contrôler le runner

Dans **Settings → CI/CD → Runners** (selon la version), le runner doit être actif, affecté au projet et utilisable par les MR issues de branches non protégées. Tous les tags du job doivent exister sur le runner ; sans tags, il doit accepter les jobs sans tags. Docker/Kubernetes nécessite une image Python approuvée ; un exécuteur shell nécessite Python déjà installé. Demandez les valeurs à l'école, sans inventer image ou tags.

02 · Le parcours de travail

300 minutes exactes · 230 minutes au cœur des trois MR, code, tests, CI et Git compris.

Créneau	Mission	Durée
0–25	Dépôt, accès, runner, règles et lecture	25 min
25–105	MR1 : validation, T1–T5, CI, rouge → vert, fusion	80 min
105–175	MR2 : connexion TCP, T6, MR et fusion	70 min
175–255	MR3 : scan, bilan, T7–T8, ZIP et fusion	80 min
255–280	Démonstration et vérification finale	25 min
280–300	Marge et débogage ciblé, bilan	20 min

Branche : ligne de travail ; **commit** : étape enregistrée.
Push : envoi à GitLab ; **MR** : demande de fusion avec diff et revue.

Le cycle à répéter trois fois

01 · Partir de la bonne base
Après chaque fusion, revenir sur la branche principale et récupérer les changements avant de créer la branche suivante.

02 · Écrire et vérifier
Développer une fonctionnalité, écrire ses tests et exécuter verification.py depuis la racine. Relire le diff, puis faire des commits qui décrivent un résultat.

03 · Ouvrir et relire la MR
Décrire le but, les changements et les tests. Une auto-revue suffit : lire le diff et les journaux du pipeline.

04 · Fusionner le dernier vert
Vérifier le pipeline de MR et son commit. Fusionner seulement si le dernier commit est vert. Ne pas créer les trois branches depuis la même ancienne base.

Visez au moins un ou deux commits utiles par fonctionnalité selon votre progression réelle. Une seule grosse MR finale ne satisfait pas ce parcours. Aucune extension pendant le créneau : après cinq heures, livrez un bilan honnête de ce qui est terminé.

03 · Les contrats du programme

Cinq fonctions · Des valeurs simples, une responsabilité par fonction.

valider_ports(ports)

Entrée : une liste. Si elle est valide, retourner une liste avec les mêmes valeurs dans le même ordre. Sinon, lever **ValueError** avec un message clair. Refuser une liste vide, plus de 10 éléments, les doublons, les valeurs non entières (booléens compris) et les ports hors de 1 à 65535. Ne pas convertir du texte en entier. En Python, bool est aussi un int : le refus des booléens doit donc être explicite. Une entrée qui n’est pas une liste est hors contrat.

tester_port(port)

Précondition : port valide. Utiliser la cible constante 127.0.0.1 et le délai 0,5 seconde. Retourner exactement **OUVERT** après connexion réussie, **REFUSE** pour ConnectionRefusedError, **TIMEOUT** pour un dépassement du délai et **ERREUR** pour un autre OSError. Toute socket créée doit être fermée, même en cas d’échec.

scanner_ports(ports)

Valider **toute la liste avant la première connexion**. Faire ensuite une seule tentative par port, dans l’ordre reçu. Retourner une liste de résultats : chaque résultat est une petite liste de deux éléments, le port puis son état. Conserver exactement l’ordre des ports.

resumer(resultats)

Retourner un dictionnaire de cinq compteurs entiers : **total, ouverts, refuses, timeouts, erreurs**. La somme des quatre catégories égale total. Une liste de résultats vide donne cinq zéros. Les résultats mal formés sont hors contrat.

main

Coordonner la liste de ports configurée dans le programme, le scan, un affichage par port et le résumé. Afficher un message explicite si la liste est invalide. Aucun rapport à écrire. L’import par verification.py ne doit pas lancer main.

Pas de saisie interactive, argparse, classes, threads, pip, installation de Docker, plages de ports, exploitation, lecture de fichiers ou scan externe. Les fonctions de données retournent leurs valeurs ; main gère l’affichage.

04 · MR1 — Valider les ports

25–105 min · Branche feat/validation · Première fonctionnalité et première CI ensemble.

Mission

Écrire valider_ports dans portpilot.py, puis cinq fonctions de test dans verification.py. Ajouter votre première .gitlab-ci.yml avec le job verifier. Il n’y a pas de MR de bootstrap séparée.

Les cinq tests à écrire

Test	Entrée / vérification attendue
T1	Liste contenant 1, 8000 et 65535 : acceptée, mêmes valeurs, même ordre.
T2	Liste vide : ValueError.
T3	Onze ports distincts, tous individuellement valides : ValueError.
T4	Une liste d’un élément pour chacun des six sous-cas : −1, 0, 65536, texte « 8000 », décimal 80,5 et booléen vrai : ValueError pour chacun.
T5	Deux fois le port 8000 : ValueError.

Rendre les tests réellement exécutables

Pour T2–T5, chaque sous-cas échoue si ValueError n’est pas levée. Recevoir ValueError valide ce sous-cas ; une autre exception est un échec. Chaque test est une fonction avec des assertions. verification.py importe votre module et appelle effectivement les cinq fonctions. Affichez les cas exécutés et un total cohérent. Une assertion échouée doit rester non masquée : le processus termine avec un code non nul. Aucun mode Python optimisé, aucun test factice.

```
python verification.py
```

Adaptez python en python3 selon l’installation. Exécutez depuis la racine du dépôt. Le job verifier lance cette même commande ; il doit échouer si le programme échoue.

Terminé quand…

T1–T5 passent localement et dans un vrai pipeline de MR ; la fonction respecte tous les refus et conserve l’ordre ; le cycle rouge/vert de la page suivante prouve le blocage CI ; la MR est relue et fusionnée au dernier commit vert.

05 · MR1 — Prouver le verrou

La configuration de la branche source suffit à lancer la première CI de MR avant sa fusion.

Écrire la politique CI, sans modèle à recopier

À la racine, créez .gitlab-ci.yml. Consultez **workflow / rules** et la variable **CI_PIPELINE_SOURCE**. Autorisez seulement la valeur **merge_request_event**, sans règle finale qui accepte tous les push. Tous les jobs héritent de cette politique : ouverture de MR → pipeline ; nouveau push sur sa branche → nouveau pipeline ; push sans MR → aucun pipeline, ce qui est normal ici.

À ce stade, un seul job : **verifier**. Utilisez **script** pour ses commandes, **image** pour Python avec un exécuteur compatible, et **tags** selon le runner de l'école. Aucun allow_failure, job manuel ou mécanisme qui neutralise une erreur. Dans **Build → Pipeline editor**, utilisez la validation (CI Lint) : elle vérifie la configuration, pas son exécution. Consultez **Build → Pipelines** (ou CI/CD selon la version) ; les résultats apparaissent aussi dans la MR. Dans GitLab, contrôlez l'étiquette « merge request » et le dernier commit testé.

Faire un cycle rouge → vert

01 · Obtenir un premier vert
Ouvrir la MR avec le programme correct et T1–T5. Vérifier les journaux et le commit du pipeline.

02 · Créer une petite régression dans le code
Par exemple, refuser à tort un port frontière pourtant valide. Garder les tests corrects. Commit et push sur la même branche.

03 · Observer la fusion bloquée
Le pipeline devient rouge. Sur une MR non-Draft sans conflit, un compte autorisé constate le blocage lié à l'échec CI. Garder le lien du pipeline et le motif affiché.

04 · Corriger le code, puis fusionner
Corriger la régression, conserver les attendus des tests, commit et push. Vérifier le dernier vert, relire le diff, puis fusionner.

La règle à retenir

Ne jamais supprimer un test correct ou modifier son attendu pour obtenir du vert. Le rouge sert à détecter une régression réelle ; la correction doit porter sur le code fautif.

06 · MR2 — Une vraie connexion

105–175 min · Branche feat/connexion · Conserver T1–T5 et ajouter T6.

Mission

Écrire tester_port. Comprendre une socket TCP IPv4, le délai de connexion et la fermeture des ressources. Ajouter un test réseau positif autonome, sans dépendre d’un serveur déjà présent sur le runner.

Les API à explorer

Pour le client, explorez **socket.create_connection** : cette API simple permet une connexion TCP avec un délai. Pour les écoutes de test : **socket.socket, AF_INET, SOCK_STREAM, bind, listen, getsockname, close**. Prévoir des exceptions précises : ConnectionRefusedError, le dépassement du délai et les autres OSError. Traiter ConnectionRefusedError et TimeoutError avant OSError : les catégories spécifiques ne doivent pas être absorbées dans le cas général.

T6 · Recette du serveur temporaire

01 · Créer l’écoute locale
Dans votre test, créer une socket TCP IPv4. Avec bind, utiliser 127.0.0.1 et le port 0 : le système choisit un port libre. Appeler listen.

02 · Trouver le port et tester
Lire le port réel avec getsockname. Garder la socket d’écoute ouverte pendant l’appel à tester_port. Vérifier que l’état retourné est exactement OUVERT.

03 · Fermer dans tous les cas
Utiliser un gestionnaire de contexte ou finally. Pas besoin d’accept ni de thread pour cette connexion unique : l’écoute permet le handshake TCP.

Le port 0 sert uniquement à allouer un port dans le test. Il reste interdit dans la liste à scanner. Ne fermez pas l’écoute avant le scan : le port pourrait être réattribué.

Terminé quand…

Le code respecte les quatre états et ferme toute socket. verification.py appelle T1–T6 ; ces six tests passent localement et dans le pipeline de MR. Vous relisez le diff et les branches d’exception, puis fusionnez au dernier commit vert.

Les écoutes de test sont locales au job CI. Elles n’utilisent aucun port de la machine de l’étudiant, aucun accès Internet, aucun sleep fixe et aucun timeout réseau volontaire.

07 · MR3 — Scanner et résumer

175–255 min · Branche feat/scan-bilan · Terminer le programme et les huit tests.

Mission

Écrire scanner_ports, resumer et main. Valider avant toute entrée/sortie, conserver l'ordre, afficher les résultats et un bilan. Ajouter T7 et T8 sans retirer les six tests existants. Finaliser README.md dans cette MR, avant le pipeline de livraison et la fusion : le ZIP doit contenir cette version finale. Choisir aussi un port de démonstration disponible et l'inscrire dans la liste configurée avant le dernier commit de MR3.

T7 · Deux connexions, dans le bon ordre

Créer deux sockets d'écoute temporaires distinctes avec la recette de T6. Les maintenir ouvertes ensemble : le système ne peut pas attribuer le même port aux deux. Appeler scanner_ports avec ces deux ports dans un ordre choisi. Vérifier exactement deux résultats OUVERT, chacun avec le bon port et dans le même ordre que l'entrée. Faire une seule connexion par écoute. Fermer les deux écoutes systématiquement.

T8 · Un bilan exact, y compris vide

Fabriquer cinq résultats au format contractuel : deux OUVERT, un REFUSE, un TIMEOUT et un ERREUR. Vérifier le dictionnaire : total 5, ouverts 2, refuses 1, timeouts 1, erreurs 1. Vérifier aussi une liste vide : les cinq compteurs valent zéro. La somme des quatre catégories doit être égale au total.

Terminé quand…

- La liste configurée respecte les bornes ; scanner_ports valide tout avant de connecter ; chaque port reçoit un seul essai séquentiel. main affiche un état par port et les cinq compteurs, ou un message clair de validation.
- Importer portpilot depuis verification.py ne déclenche ni scan ni affichage du programme. Les huit fonctions de test sont toutes appelées, avec un total cohérent.
- Le pipeline teste, construit l'artefact décrit page suivante, puis devient vert. La MR est relue et fusionnée sur le dernier commit testé.

Ce que le socle ne prouve pas

T6 et T7 automatisent le succès réseau et l'ordre. T8 compte des états fabriqués ; il ne provoque pas les erreurs réseau. REFUSE, TIMEOUT et ERREUR doivent être implémentés et relus, sans prétendre à une couverture totale. Les tests négatifs ou le transport simulé sont des bonus après les cinq heures.

08 · MR3 — Livrer le ZIP

Un paquet du programme testé · Aucun déploiement serveur.

Étendre votre CI seulement maintenant

Définir deux stages successifs : **test**, puis **livraison**. verifier exécute verification.py. Ajouter le job **livrer** dans le second stage : il construit l'archive uniquement après le succès de verifier. Conserver la politique de pipelines de MR de la MR1.

Le contenu exact à produire

Archive	Fichiers à la racine
portpilot.zip	portpilot.py verification.py README.md

Utiliser le module standard zipfile ; aucun paquet à installer. Ne pas inclure .git, les caches, .gitlab-ci.yml, .gitignore ni l'archive elle-même. Le ZIP est un artefact téléchargeable, avec une conservation demandée de **7 jours**. Consultez les mots-clés artifacts, paths et expire_in pour écrire votre configuration.

Commandes d'outils autorisées

```
python -m zipfile -c portpilot.zip portpilot.py verification.py README.md
```

```
python -m zipfile -l portpilot.zip
```

La première commande crée le paquet ; la seconde permet de contrôler ses trois entrées. Adaptez l'interpréteur à l'école. Ces outils ne remplacent ni votre programme ni votre configuration YAML.

Vérifier la livraison

Ouvrir le dernier pipeline vert de MR3, télécharger portpilot.zip et vérifier son contenu. Il correspond au commit testé par ce pipeline. Le parcours utilise uniquement des pipelines de MR : aucun pipeline après fusion n'est imposé.

Conserver la preuve

Garder le lien du pipeline final et télécharger l'archive avant son expiration. L'énoncé ne fournit aucun ZIP prêt à l'emploi : c'est votre CI qui produit la livraison.

09 · Les commandes Git utiles

Remplacer les éléments entre chevrons · Garder les noms de branche du parcours.

Le nom de la branche principale peut être main ou master : vérifiez celui du projet. Les chevrons ci-dessous sont des indications à remplacer, pas à saisir tels quels. À chaque étape, lire git status et git diff avant de préparer le commit.

Démarrer depuis le dépôt préparé

```
git clone <URL_DU_DEPOT>
```

Entrez ensuite dans le dossier cloné avec votre terminal.

```
git status
```

```
git switch -c feat/validation
```

MR1 : enregistrer votre travail puis ouvrir la MR

```
git diff
```

```
git add portpilot.py verification.py .gitlab-ci.yml
```

```
git commit -m "Valide les ports et teste les cas invalides"
```

```
git push -u origin feat/validation
```

Ouvrir ensuite la MR dans GitLab vers la branche principale. Pour chaque correction sur cette branche : relire le diff, ajouter les fichiers modifiés explicitement, commit, puis git push. La MR reçoit un nouveau pipeline.

Après chaque fusion : actualiser la base

```
git switch <BRANCHE_PRINCIPALE>
```

```
git pull --ff-only
```

```
git switch -c feat/connexion
```

Pour MR2, ajouter explicitement portpilot.py et verification.py ; commit descriptif, puis :

```
git push -u origin feat/connexion
```

Après MR2 fusionnée, répéter switch vers la principale et pull, puis :

```
git switch -c feat/scan-bilan
```

Développer MR3, tester, lire git diff, ajouter explicitement les fichiers modifiés (dont README.md et .gitlab-ci.yml), puis faire un commit descriptif. Évitez git add . : archives et fichiers temporaires restent hors des commits.

```
git push -u origin feat/scan-bilan
```

Ouvrir ensuite la troisième MR dans GitLab.

10 · Démontrer et rendre

255–280 min · Puis 20 minutes de marge pour corriger et faire le bilan.

Une démonstration locale contrôlée

Utiliser le port disponible choisi et configuré avant le dernier commit de MR3, sans modifier le programme livré. Dans un premier terminal, lancer le serveur sur ce port ; exemple si vous aviez choisi 8000 :

```
python -m http.server 8000 --bind 127.0.0.1
```

Dans un second terminal, exécuter PortPilot avec sa liste déjà livrée :

```
python portpilot.py
```

Le port doit apparaître OUVERT. Arrêter le serveur avec Ctrl+C et relancer PortPilot : REFUSE est attendu dans ce laboratoire contrôlé. S’il reste ouvert, rechercher un autre processus en écoute. Cette démonstration n’envoie aucune requête HTTP : elle teste seulement TCP. Le port fixe ne sert jamais aux tests CI.

README : court, mais utilisable

Finalisé dans MR3 avant sa livraison, le README doit documenter le but et la limite locale, les prérequis Python/GitLab, la façon de modifier la liste des ports, le lancement, les quatre états, la commande des tests et le téléchargement de l’artefact. Préciser le délai, les bornes et la limite de couverture des erreurs réseau.

La remise attendue

- Dépôt GitLab contenant vos cinq fichiers : portpilot.py, verification.py, README.md, .gitignore et .gitlab-ci.yml.
- Trois MR fusionnées, leurs trois derniers pipelines verts, et le lien du rouge MR1 avec le motif de blocage puis la correction verte.
- Huit fonctions de test appelées à la fin ; portpilot.zip téléchargeable depuis le dernier pipeline vert de MR3, avec exactement les trois fichiers demandés.

Barème sur 20

Critère	Points
Python fonctionnel et contrats respectés	8
Tests pertinents et réellement appelés	4
Trois MR et CI réellement bloquante	6
Lisibilité et README	2

11 · Aide et références

Documentation officielle à consulter au moment où la question se pose.

Une revue courte avant chaque fusion

Noms clairs ; constantes HOST, TIMEOUT et MAX_PORTS ; une responsabilité par fonction ; pas de duplication ; validation avant connexion ; sockets fermées ; exceptions précises ; fonctions de données qui retournent des valeurs ; main pour l’affichage ; import sans exécution. Lire le diff et conserver les attendus corrects des tests.

Déboguer sans élargir le projet

Pas de pipeline ? Vérifier que la MR est ouverte et la politique merge_request_event. Job en attente ? Vérifier runner, tags et accès aux branches non protégées. Job rouge ? Lire le premier échec, reproduire localement, corriger le code ou une configuration réellement fautive. CI Lint ne garantit pas un runner disponible.

Gardez le périmètre : pas d’extensions pendant cinq heures. L’objectif est un premier outil maîtrisé et une livraison par MR, pas une expertise cyber en une séance.

GitLab : pipelines et fusion

Pipelines de merge request

Exiger un pipeline réussi pour fusionner

Protéger les branches

Règles des jobs et prévention des pipelines en double

Référence YAML : script, image, tags, stages et artifacts

Python et Git : API et outils

socket : connexions, écoutes et délais

Exceptions intégrées : ValueError et OSError

__main__ : point d’entrée et import

assert : vérifier une condition

zipfile : interface en ligne de commande

Git : switch

Git : pull